

LAPORAN AKHIR PENGEMBANGAN SISTEM RAG CHATBOT

Pembuatan Sistem Retrieval-Augmented Generation dengan n8n, Pinecone, dan Cloudflare

Identitas Laporan

Nama: Mahendra Zidane Rainafa

NIM: 24523142

Mata Kuliah: Sistem Jaringan Komputer

BAB I. RINGKASAN INFRASTRUKTUR SISTEM

1.1 Evolusi Arsitektur

Pengembangan sistem ini dimulai dengan infrastruktur lokal sederhana menggunakan Docker untuk menjalankan layanan n8n. Pada tahap awal (Progress 1-5), akses publik dilakukan menggunakan **Ngrok**. Namun, Ngrok memiliki keterbatasan pada stabilitas URL (berubah saat restart) dan keamanan yang minim.

Pada tahap akhir (Progress 6), infrastruktur bermigrasi total ke **Cloudflare Zero Trust** (Tunnel). Perubahan ini memberikan stabilitas dengan domain kustom (*.my.id*), enkripsi SSL otomatis, dan kemampuan untuk menempatkan layanan di belakang Firewall tanpa membuka port pada router fisik.

1.2 Komponen Infrastruktur (Docker & Node.js)

Sistem dijalankan dalam lingkungan terkontainerisasi menggunakan *docker compose*, yang terdiri dari tiga layanan utama:

1. **n8n (Workflow):** Pusat logika backend dan manajemen AI.
2. **WebApp Proxy (Node.js):** Bertindak sebagai *Secure Proxy* di port 3000. Layanan ini menyembunyikan endpoint webhook n8n yang asli, memberikan lapisan keamanan tambahan sebelum data diteruskan ke backend.

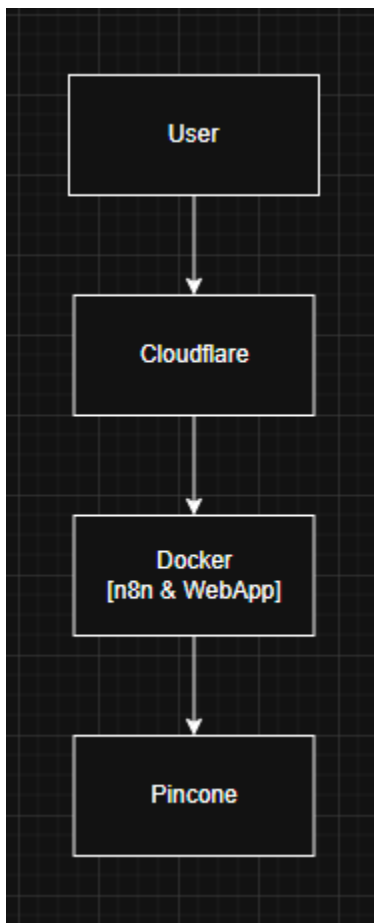
3. **Cloudflared (Tunnel Daemon):** Menghubungkan container lokal ke jaringan edge Cloudflare secara aman.
4. **Pinecone (Vector Database):** Tempat penyimpanan permanen bagi "memori" chatbot. Pinecone menyimpan vektor hasil embedding dari dokumen PDF dan melakukan pencarian kemiripan (*similarity search*) saat user bertanya.

NAME	IMAGE	COMMAND	SERVICE	CREATED	STATUS	P
cloudflared-tunnel	cloudflare/cloudflared:latest	"cloudflared --no-au..."	cloudflared	4 hours ago	Up 4 hours	
nodejs-webapp	project-rag-webapp	"docker-entrypoint.s..."	webapp	3 hours ago	Up 3 hours	0
project-rag-n8n-1	n8nio/n8n	"tini -- /docker-ent..."	n8n	4 hours ago	Up 4 hours	0

Gambar 1.2 Status Container Docker yang berjalan aktif.

1.3 Diagram Arsitektur Sistem Akhir

Untuk memberikan gambaran menyeluruh tentang bagaimana data mengalir dalam sistem, berikut adalah diagram arsitektur akhir proyek ini. Diagram ini menunjukkan integrasi antara pengguna (User), lapisan keamanan (Cloudflare), infrastruktur kontainer (Docker), dan layanan AI eksternal (Groq & Pinecone).



Gambar 1.3 Diagram Arsitektur End-to-End Sistem RAG.

BAB II. IMPLEMENTASI WORKFLOW INTI

Pengembangan logika kecerdasan buatan (AI) dilakukan secara bertahap menggunakan n8n sebagai orkestrator. Sistem dibagi menjadi empat workflow terpisah untuk memvalidasi kemampuan model bahasa dasar (*Basic LLM*) sebelum ditingkatkan menjadi sistem RAG yang kompleks.

2.1 Workflow 1: Telegram AI Agent (Basic LLM)

Workflow ini dirancang sebagai tahap awal untuk menguji konektivitas antara platform Telegram dan model bahasa Groq (Llama-3.1). Pada tahap ini, AI bekerja sepenuhnya mengandalkan **Parametric Memory** (Memori Parametrik), yaitu pengetahuan bawaan yang diperoleh model selama proses pelatihan awal (*pre-training*).

1. Input (Trigger):

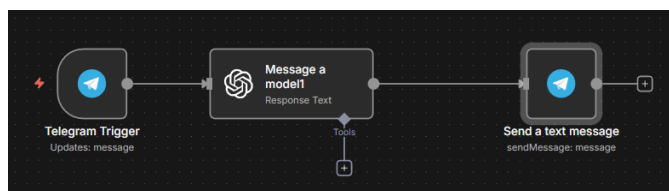
- Pengguna mengirim pesan teks melalui bot Telegram.
- Node *Telegram Trigger* di n8n menangkap pesan tersebut beserta metadata (Chat ID, First Name).

2. Proses (Inferensi AI):

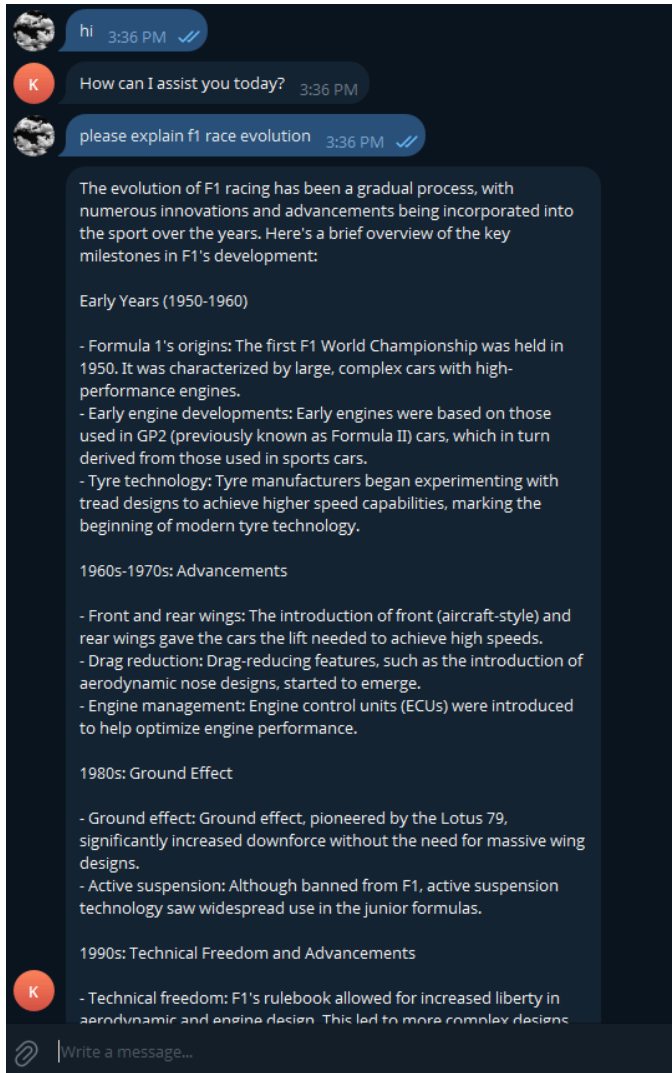
- Pesan diteruskan ke node OpenAI Model (*Groq credential*)
- Karena belum terhubung ke database eksternal, model menjawab pertanyaan berdasarkan pola statistik dan data pelatihan internalnya (General Knowledge). Model dapat menjawab berbagai topik umum tanpa batasan konteks spesifik.

3. Output (Response):

- Hasil teks dari node OpenAI model dikirim kembali ke node *Telegram Sender*.
- Bot mengirimkan balasan ke pengguna di aplikasi Telegram.



Gambar 2.1.1 Workflow Telegram Bot berbasis Memori Parametrik.



Gambar 2.1.2 Telegram chat berhasil.

2.2 Workflow 2: WebApp AI Agent (Frontend Integration)

Workflow ini bertujuan memvalidasi arsitektur *Client-Server* antara WebApp (Vercel) dan Backend (n8n). Sama seperti workflow sebelumnya, agen ini menggunakan **Parametric Memory** untuk memberikan respons umum terhadap input pengguna dari antarmuka web.

1. Input (HTTP Request):

- Pengguna mengetik pesan di WebApp dan menekan tombol kirim.
- Frontend mengirimkan HTTP POST Request ke endpoint Cloudflare Tunnel yang diteruskan ke node *Webhook* di n8n. Format data berupa JSON.

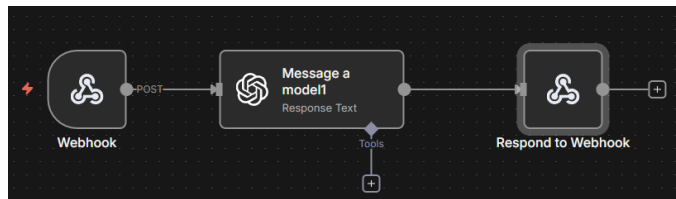
2. Proses (Inferensi AI):

- Node OpenAI Model (*Groq credential*) menerima prompt dari body JSON.

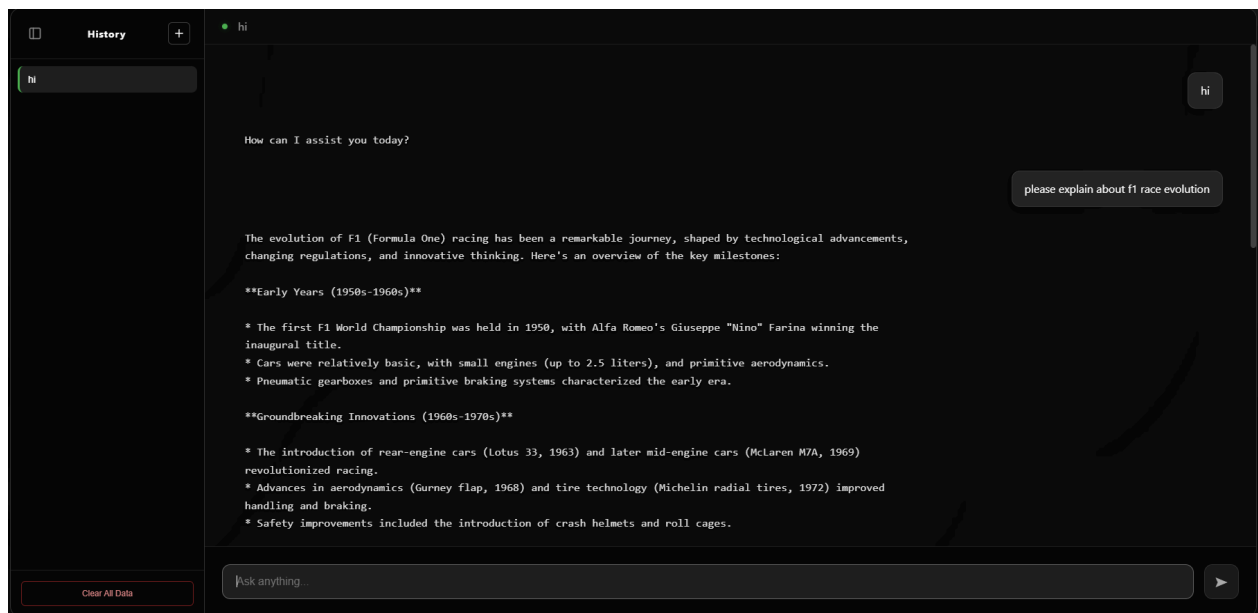
- Model memproses input menggunakan pengetahuannya (Llama-3.1) untuk menyusun jawaban yang koheren secara tata bahasa dan konteks umum.

3. Output (JSON Response):

- Jawaban AI dikemas kembali dalam format JSON oleh node *Respond to Webhook*.
- Frontend menerima respons tersebut dan menampilkannya di layar chat pengguna.



Gambar 2.2.1 Integrasi WebApp dengan AI menggunakan Webhook.



Gambar 2.2.2 Webapp chat berhasil. .

2.3 Workflow 3: Document Ingestion (ETL Pipeline)

Workflow ini tidak melibatkan interaksi pengguna, melainkan proses **ETL (Extract, Transform, Load)** untuk membangun basis pengetahuan eksternal (*Non-Parametric Memory*) dari dokumen PDF.

1. Input (Source Data):

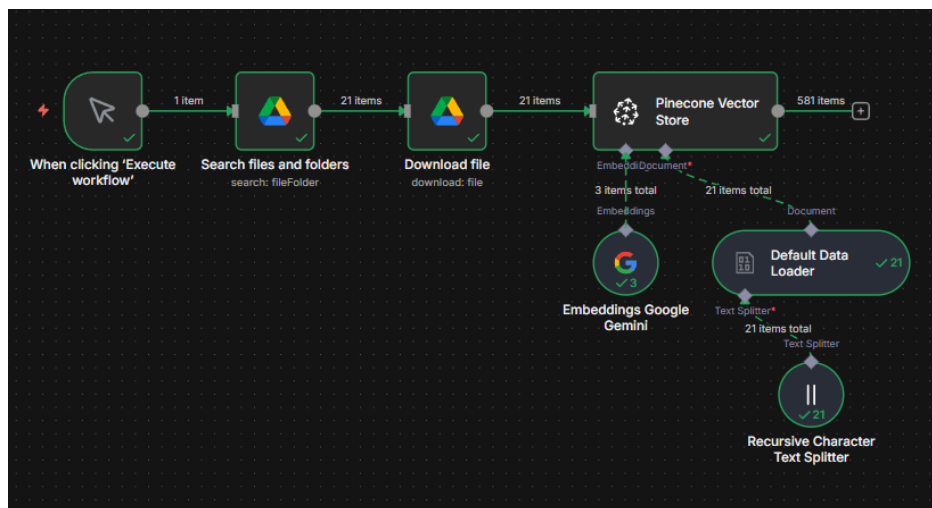
- Node *Google Drive Trigger* mendeteksi atau mengambil file PDF target yang berisi materi pengetahuan spesifik.

2. Proses (Transformasi & Embedding):

- **Extraction:** Node *Read Binary File* mengekstrak teks mentah dari dokumen PDF.
- **Chunking:** Teks panjang dipecah menjadi potongan-potongan kecil (chunks) menggunakan *Character Text Splitter* agar muat dalam jendela konteks model.
- **Embedding:** Setiap potongan teks dikirim ke model **Google Gemini Embedding**. Model ini mengonversi teks menjadi array vektor (deretan angka) yang merepresentasikan makna semantik kalimat tersebut.

3. Output (Vector Storage):

- Vektor hasil embedding beserta metadata teks aslinya disimpan ke dalam database **Pinecone**. Data ini siap dicari kembali (*retrievable*) di workflow selanjutnya.



Gambar 2.3 Pipeline ETL untuk konversi Dokumen ke Vektor.

2.4 Workflow 4: Full Integration (RAG System)

Ini adalah implementasi final yang menggabungkan seluruh komponen. Workflow ini mengubah cara kerja AI dari sekadar mengandalkan ingatan hafalan (Parametric) menjadi berbasis referensi data (RAG).

1. Input (Multi-Channel):

- Sistem menerima trigger dari dua sumber sekaligus: Telegram (Pesan Chat) atau WebApp (Webhook).

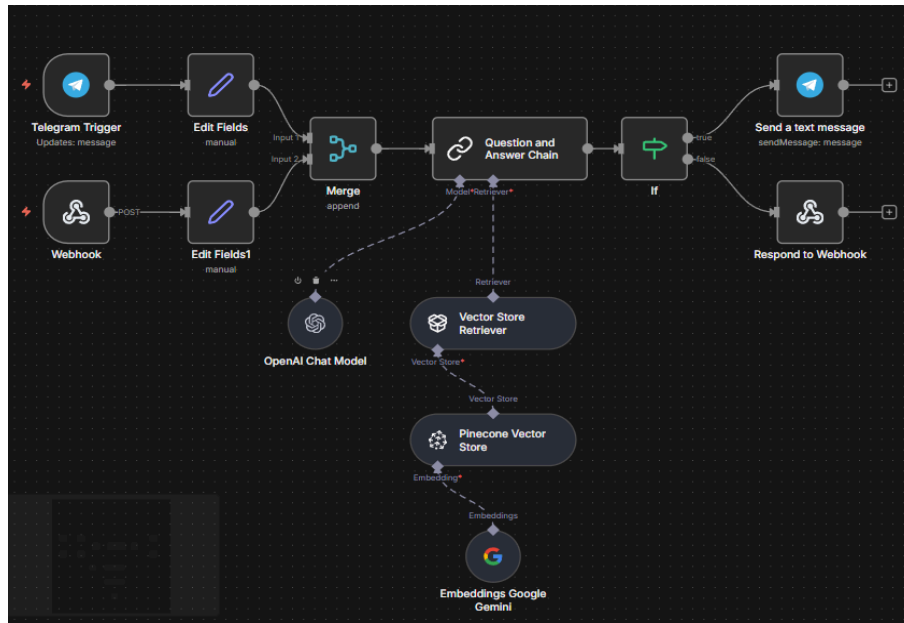
2. Proses (Retrieval & Augmentation):

- **Query Embedding:** Pertanyaan pengguna diubah menjadi vektor menggunakan Google Gemini Embedding.
- **Similarity Search:** Vektor pertanyaan dicocokkan dengan database Pinecone untuk mencari dokumen yang memiliki kemiripan makna (*Semantic Search*).
- **Prompt Engineering:** Potongan teks relevan yang ditemukan di Pinecone disisipkan ke dalam "System Prompt" model Groq. Instruksi diubah menjadi:

"Jawab pertanyaan pengguna hanya berdasarkan konteks berikut: [Data dari Pinecone]".

3. Output (Context-Aware Response):

- Groq menghasilkan jawaban yang akurat sesuai fakta dalam dokumen PDF.
- Jawaban dikirimkan kembali ke platform asal (Telegram atau WebApp) sesuai sumber request.



Gambar 2.4 Arsitektur RAG yang menggabungkan Memori Parametrik dan Konteks Vektor.

BAB III. INTEGRASI END-TO-END SISTEM RAG

Integrasi sistem telah berhasil menghubungkan seluruh komponen terpisah menjadi satu kesatuan fungsi. Frontend (Vercel) berkomunikasi dengan Backend (n8n lokal) melalui jalur aman Cloudflare Tunnel.

Database Pinecone berfungsi sebagai "otak" jangka panjang, memungkinkan Chatbot (baik di Telegram maupun Web) memberikan jawaban yang konsisten dan akurat berdasarkan dokumen referensi yang telah diunggah.

rag-project •

METRIC	DIMENSIONS	HOST
cosine	768	https://rag-project-3do6c5o.svc.aped-4627-b74a.pinecone.io

CLOUD	REGION	TYPE	CAPACITY MODE	RECORD COUNT
aws AWS	us-east-1	Dense	On-demand	581

Gambar 3.1 Dashboard Pinecone yang berisi data vektor dokumen.

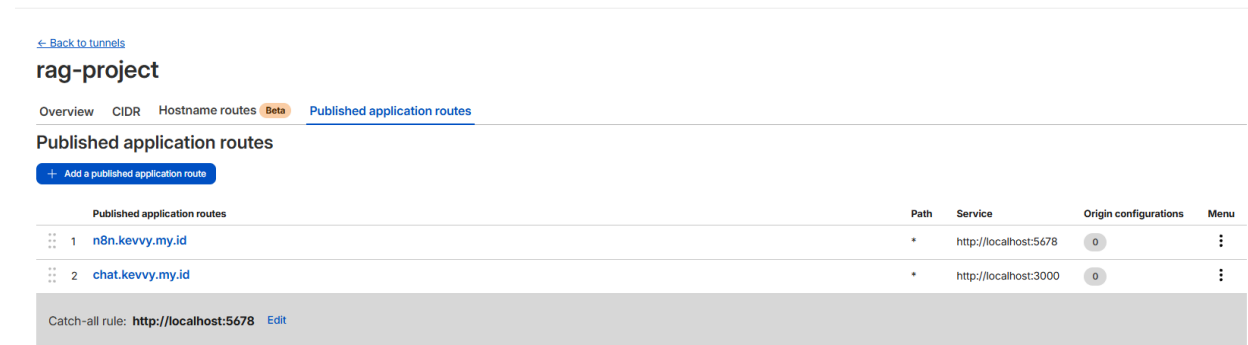
BAB IV. KEAMANAN DAN INFRASTRUKTUR CLOUD

Bagian ini merinci peningkatan keamanan yang diterapkan pada Progress 6 untuk memenuhi standar deployment produksi.

4.1 Cloudflare Tunnel & Routing

Menggunakan teknologi Zero Trust, sistem tidak lagi memerlukan *Port Forwarding* yang berisiko. Routing dikonfigurasi melalui *tunnel-config.yaml* dengan pembagian akses:

1. **Admin Dashboard:** *n8n.kevvv.my.id* → *localhost:5678*
2. **Public User Access:** *chat.kevvv.my.id* → *localhost:3000* (Secure Proxy)



Gambar 4.1 Konfigurasi Ingress Rules pada Cloudflare Tunnel.

4.2 Web Application Firewall (WAF)

Untuk melindungi API dari serangan bot dan penyalahgunaan kuota AI, diterapkan 2 aturan Firewall (WAF) ketat:

1. **Rule: HTTP Method Filtering**
 - **Mekanisme:** Hanya mengizinkan metode *GET*, *POST*, dan *OPTIONS*.
 - **Tujuan:** Mencegah serangan manipulasi data menggunakan metode HTTP yang tidak standar.
2. **Rule: Trusted Referer Enforcement (Whitelisting)**
 - **Mekanisme:** Memblokir semua akses ke endpoint */api/chat* kecuali permintaan tersebut berasal dari domain frontend resmi (*kevvv.vercel.app*).
 - **Tujuan:** Mencegah akses langsung via API Tools (Postman) atau hotlinking.

Security

Security rules

Secure your domain with security actions that run on incoming requests. Configure both Cloudflare's managed rules and your own custom security rules. You can build custom security rules from scratch or use templates to help you get started.

[Security rules documentation](#) [Traffic sequence](#)

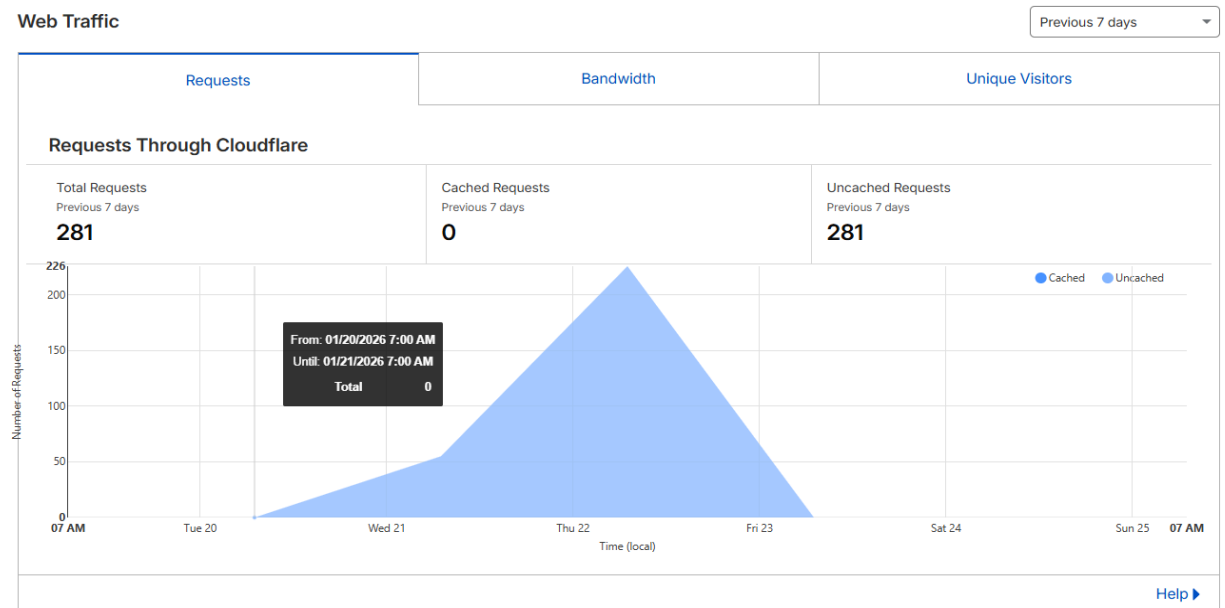
Security rules DDoS protection

Order	Name	Match against	Action	CSR ⓘ	Events last 24h	
1	Block Bad Request	Request Method is not in GET, POST, OPTIONS	Block	-	0	Active
2	Trusted Referer Enforcement	URI Path contains /api/chat and Referer does not contain kevy.vercel.app and Referer does not contain...	Block	-	2	Active

Gambar 4.2 Implementasi Aturan Firewall (WAF) untuk keamanan API.

4.3 Monitoring & Analytics

Seluruh trafik yang masuk ke sistem dipantau melalui dashboard Cloudflare, memungkinkan deteksi dini terhadap lonjakan trafik mencurigakan atau serangan DDoS.



Gambar 4.3 Monitoring trafik jaringan real-time.

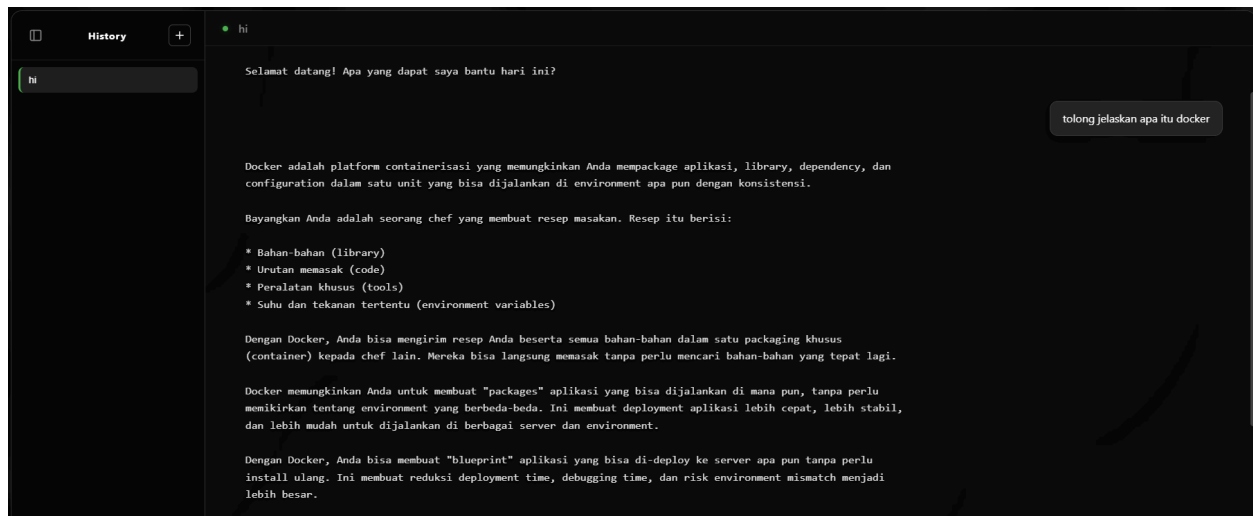
BAB V. KONFIGURASI DNS DAN DOMAIN

Pengujian dilakukan untuk memverifikasi fungsionalitas dan keamanan sistem.

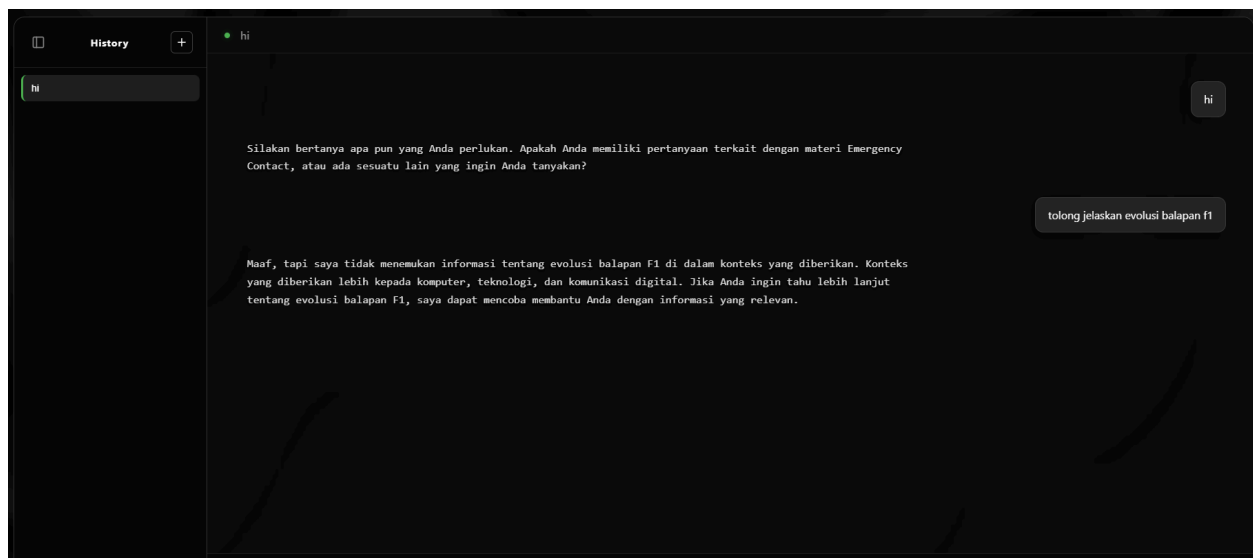
5.1 Validasi Fungsional (RAG Chatbot)

Pengujian fungsional dilakukan untuk memastikan sistem dapat menjawab pertanyaan spesifik berdasarkan dokumen PDF yang telah di-ingest ke Pinecone, baik melalui WebApp maupun Telegram.

1. **Pengujian WebApp** Chatbot melalui WebApp berhasil menjawab pertanyaan teknis tentang "Docker" dengan akurat sesuai materi, membuktikan integrasi Backend dan Frontend berjalan lancar.

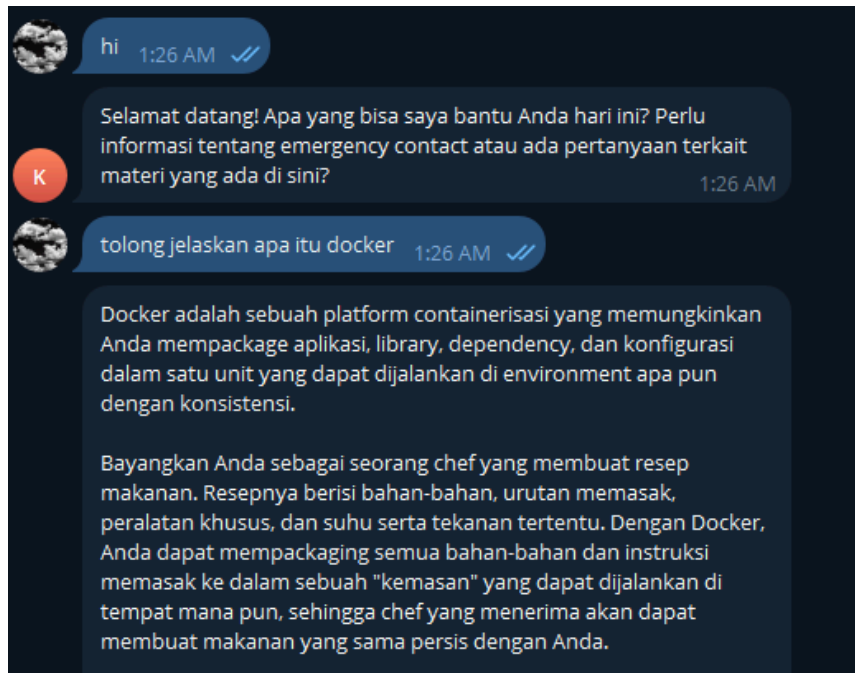


Gambar 5.1.1 Chatbot berhasil menjawab pertanyaan materi sesuai dengan data pinecone.

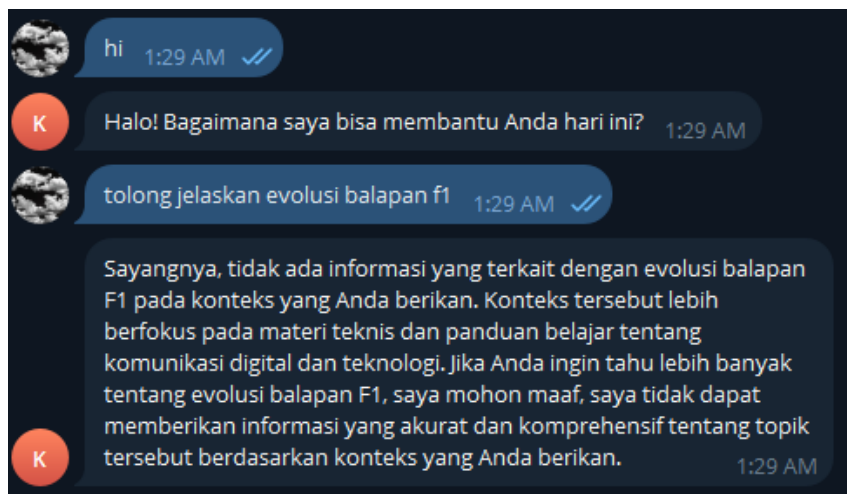


Gambar 5.1.2 Chatbot gagal menjawab karena tidak sesuai data pinecone.

2. **Pengujian Telegram Bot** Sistem juga diuji melalui platform Telegram. Bot berhasil mengambil konteks dari vektor database Pinecone dan memberikan jawaban yang relevan, membuktikan bahwa workflow integrasi (Workflow 4) berhasil melayani *multi-channel support*.



Gambar 5.1.3 Chatbot Telegram berhasil menjawab pertanyaan materi sesuai dengan data pinecone.

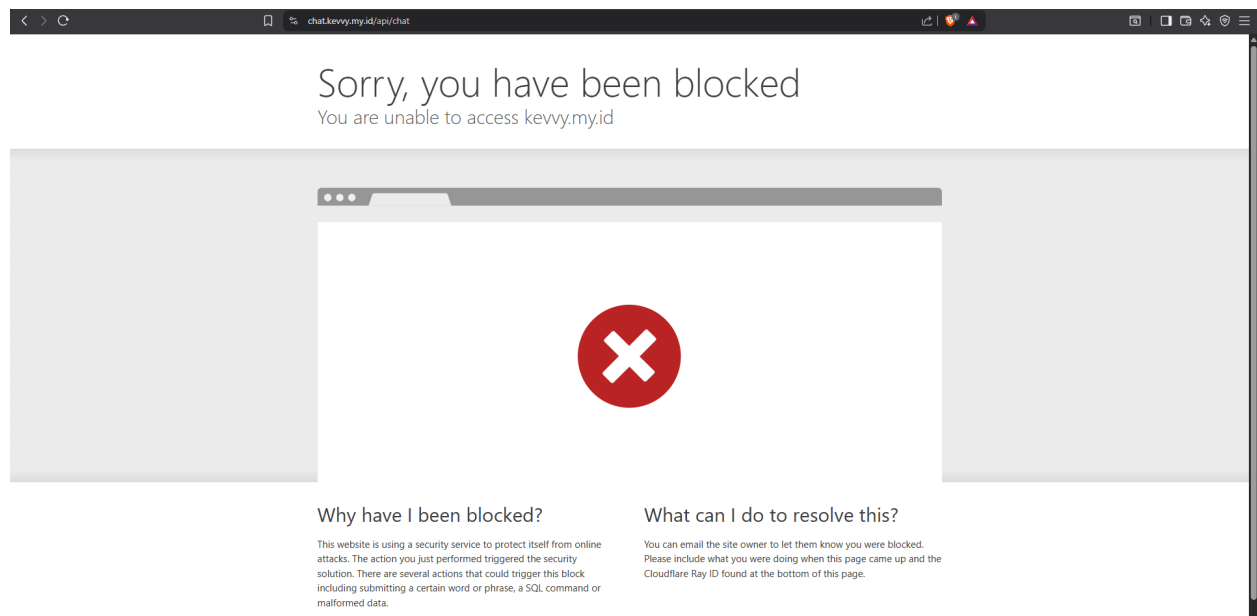


Gambar 5.1.4 Chatbot Telegram gagal menjawab karena tidak sesuai data.pinecone.

5.2 Validasi Keamanan

Pengujian keamanan dilakukan dengan mencoba mengakses endpoint API (<https://chat.kevvy.my.id/api/chat>) secara langsung melalui browser tanpa melalui frontend

resmi. **Hasil:** Cloudflare WAF berhasil memblokir akses tersebut (Sorry, you have been blocked), membuktikan bahwa proteksi "Referer Validation" bekerja dengan baik.



Gambar 5.2 Bukti Cloudflare WAF memblokir akses ilegal (Direct Access).

LAMPIRAN ARTEFAK

Berikut adalah tautan ke sumber daya proyek:

1. **Repository GitHub:** <https://github.com/kevy01/Project-RAG.git>
2. **Live Demo WebApp:** <https://kevy.vercel.app/>
3. **Telegram Bot:** <https://t.me/Kevvy01Bot>
4. **Endpoint API:** <https://chat.kevy.my.id/api/chat>

Demikian laporan akhir ini disusun sebagai bukti penyelesaian seluruh tahapan proyek RAG.